Technical Report for 3D Scanner Project

1) Technical Summary of Project Objectives

Overall Goal:

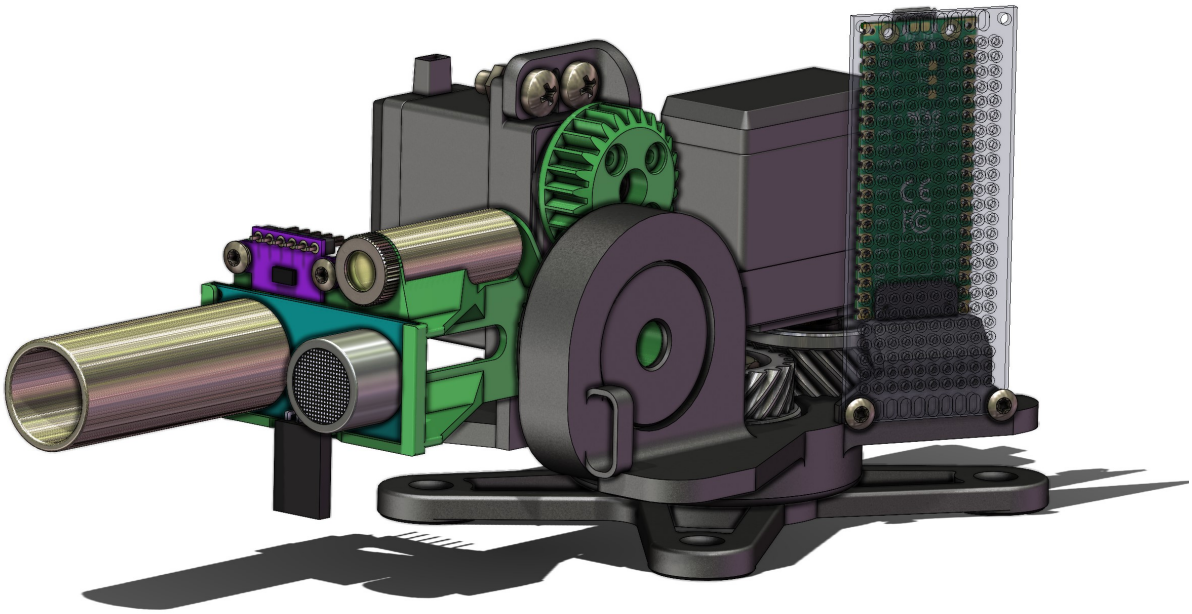
The primary goal of this project was to develop a functional 3D scanner capable of capturing spatial data of objects with as high precision as possible using basic sensors like ultrasonic sensors and Time-of-Flight (ToF) sensors, as opposed to more advanced technologies like LIDAR. Recognizing the inherent limitations of ultrasonic sensors in terms of precision and accuracy, the project served as a learning exercise to explore the feasibility and challenges of 3D scanning with crude measuring devices. Emphasis was placed on creating a scalable system that seamlessly integrates with Python-based data visualization tools. The scanner aimed to map the surfaces of objects as accurately as possible by measuring distances from various angles and compiling the data into a 3D model.

Purpose and Design Requirements:

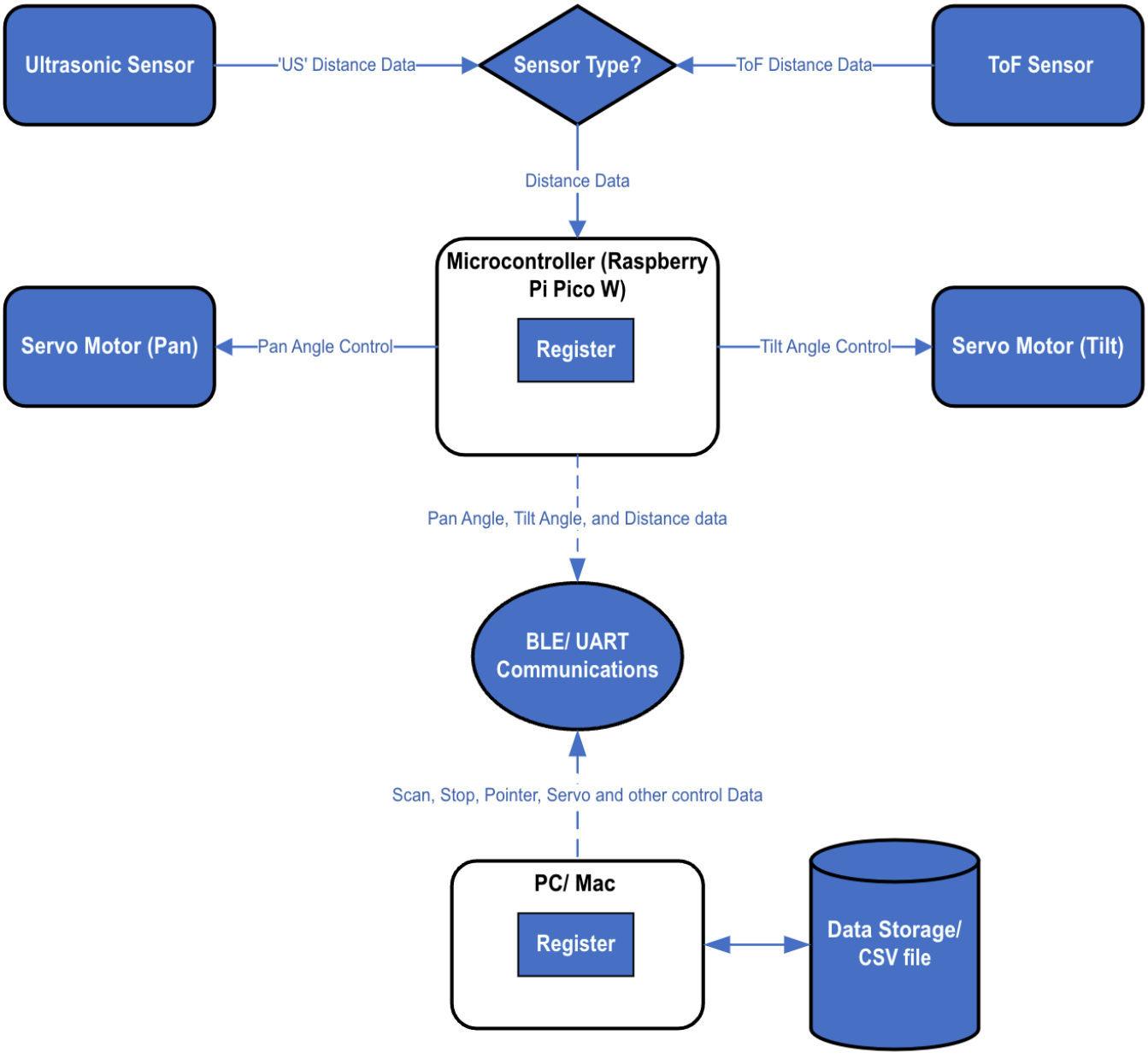
To achieve this, specific requirements were established. The project utilized ultrasonic sensors for distance measurements while acknowledging their limitations compared to LIDAR. Precise pan and tilt mechanisms were developed using servo motors to position the sensor at various angles as accurately as possible. This included a careful consideration to the mechanical aspects of the project; a robust mechanical framework was constructed using 3D resin-printed parts, steel sealed ball bearings, and gear coupling mechanisms to enhance stability and reduce mechanical errors. A Python-based graphical user interface (GUI) was created for controlling the scanning process and visualizing data, enhancing the user experience. Wireless communication was enabled between the microcontroller and the PC/Mac using Bluetooth Low Energy (BLE) to eliminate cable clutter and increase flexibility from where the user can opereate from.

Expected Outcomes:

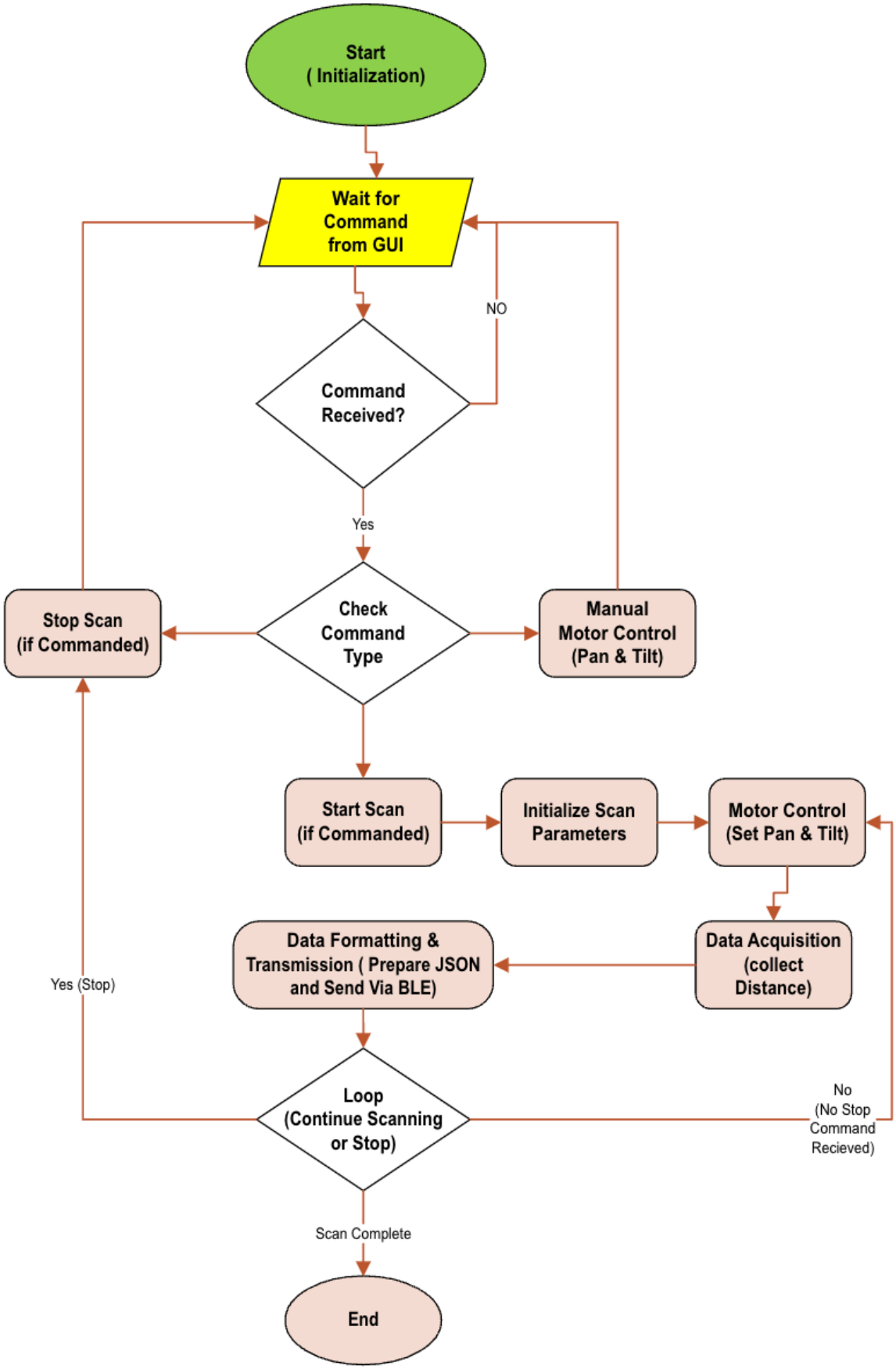
To be honest, my expectations for this project's outcomes weren't especially high. There were a lot of challenges to address—mechanical, electronic, software, and functionality issues—without a very clear picture of how it would all look or operate in the end. The main project goals included generating 3D mapping data that, while not expected to be highly accurate, would still provide valuable insights into what ultrasonic sensors can and can't do in 3D scanning. The project was intended to be a significant learning experience in hardware and software integration, working toward a functional 3D scanner as a starting point for future upgrades and experimentation. Real-time visualization of the scanned data in Python was also expected to give a better understanding of data processing and visualization.



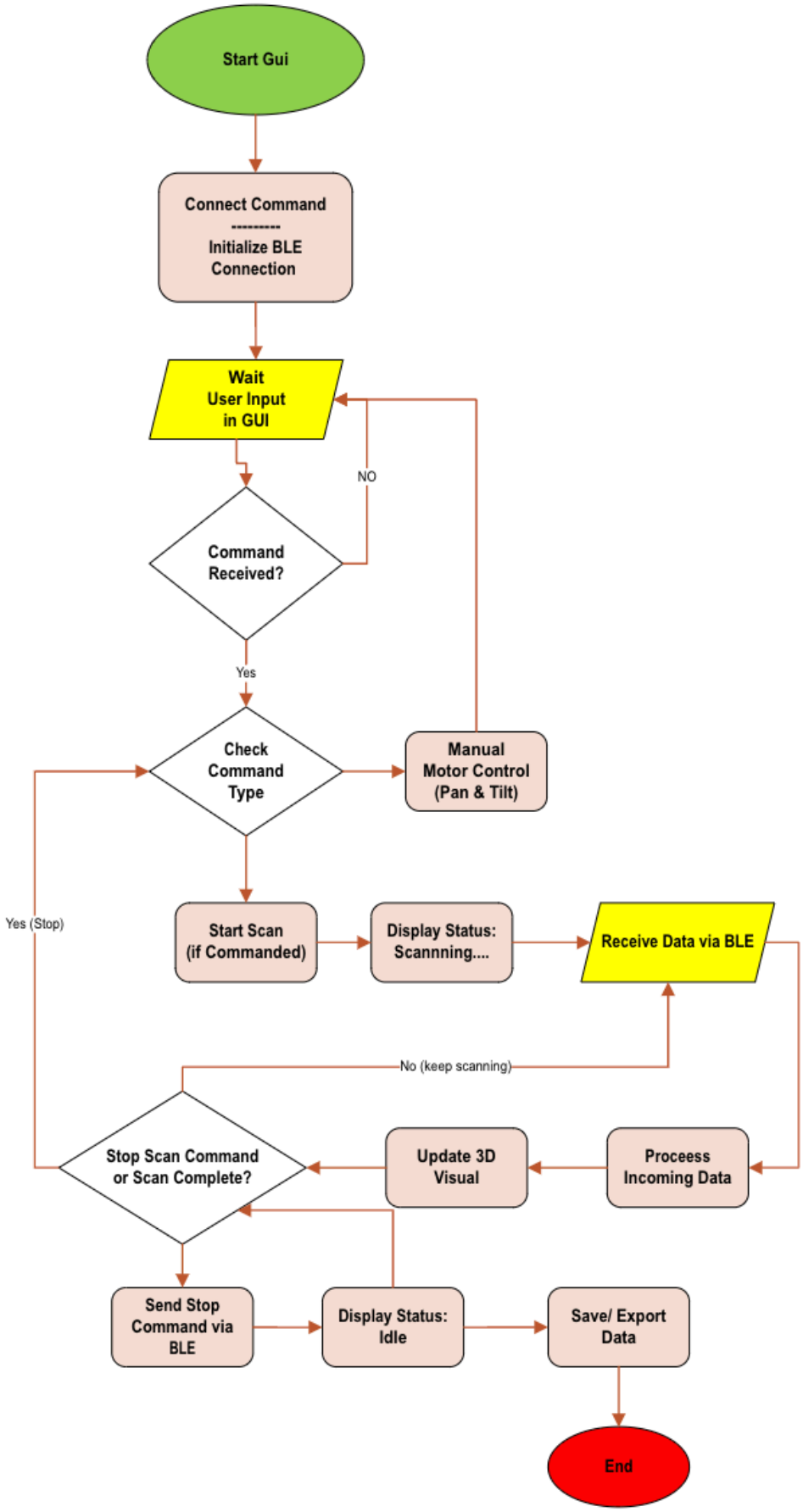
2) Block Diagram of Hardware Components



3) Block Diagram of Pico Software



4) Block Diagram of Python GUI



5) Commands Used to Send Instructions from Python GUI to Pico

The communication between the Python GUI and the Pico was facilitated through JSON-formatted commands sent over BLE. Key commands included:

- **start_scan**: Initiated the scanning process with specified pan and tilt ranges and sensor type:
{"command": "start_scan", "pan_range": [0, 180], "tilt_range": [0, 90], "sensor": “ultrasonic"}

- **stop_scan**: Stopped the scanning process.
{"command": “stop_scan"}

- **move_pan:** Moved the pan servo to a specified angle.
{"command": "move_pan", "angle": 90}

- **move_tilt:** Moved the tilt servo to a specified angle.
{"command": "move_tilt", "angle": 45}

These commands were sent asynchronously using the Bleak library in Python, allowing for real-time control of the scanning process.

6. Description of How Python Generates Instructions

The Python GUI captured user inputs such as pan and tilt ranges and sensor selection through the interface. These inputs were then formatted into JSON commands, which were sent to the Pico over BLE using the Bleak library. The transmission was handled asynchronously to ensure that the GUI remained responsive and that commands were sent without delay.

#An example code snippet demonstrating how a command was sent is as follows:

```
import json
from bleak import BleakClient

command = {
    "command": "start_scan",
    "pan_range": [0, 180],
    "tilt_range": [0, 90],
    "sensor": "ultrasonic"
}

command_json = json.dumps(command)

async def send_command(address, command_json):
    async with BleakClient(address) as client:
        await client.write_gatt_char(CHARACTERISTIC_UUID, command_json.encode())
```

This approach ensured that commands were accurately formatted and transmitted, allowing the Pico to execute the desired actions promptly.

7. How Data is Returned from Pico to Python (Pico End)

On the Pico’s end, data acquisition involved reading distance measurements from the ultrasonic sensor and recording the current pan and tilt angles. The data was formatted into JSON packets, including timestamps, angles, and distance measurements.

#An example packet might look like:
{"timestamp": "2024-10-25T12:00:00Z", "pan_angle": 90, "tilt_angle": 45, "distance": 150}

These data packets were sent to the PC via BLE notifications, with the transmission handled asynchronously to prevent any interference with motor control operations. Error handling mechanisms were implemented to manage retries and acknowledgments, ensuring reliable communication even in the presence of potential connectivity issues.

8. How Data is Processed and Displayed in Python (Python End)

Upon receiving data from the Pico, the Python GUI subscribed to BLE notifications and parsed the incoming JSON packets to extract angles and distance measurements. The data was then transformed from spherical coordinates to Cartesian coordinates using the following conversion formulas:

```
def calculate_xyz(self, pan_angle, adjusted_tilt_angle, distance):
    try:
        # Convert Pan & Tilt angles to radians
        pan_rad = math.radians(pan_angle)
        tilt_rad = math.radians(-adjusted_tilt_angle)  # Negative sign for downward tilt

        # Spherical to Cartesian conversion
        x = distance * math.sin(pan_rad) * math.cos(tilt_rad)
        y = distance * math.cos(pan_rad) * math.cos(tilt_rad)
        z = distance * math.sin(tilt_rad) + self.scanner_height

        return x, y, z
    except Exception as e:
        print(f"Error in calculate_xyz: {e}")
        traceback.print_exc()
        return 0, 0, 0  # Return zeros in case of error
```

Basic error-handling and outlier detection were implemented to filter and validate the data, enhancing the accuracy of the visualization. The transformed data was then visualized using Matplotlib's mpl_toolkits.mplot3d for 3D plotting, with Plotly integrated for enhanced interactive visualization features. Real-time updates allowed users to see the scanned data as it was collected, and interactive controls enabled users to rotate, zoom, and pan the 3D model. Any data inconsistencies or errors were promptly notified to the user through the GUI.

9. Summary of Measurements, Analysis, and Conclusions

Measurements

The project involved collecting distance measurements from the ultrasonic sensor, fully aware of the inherent limitations in precision due to the sensor's crude nature. Simple objects such as a ball or shoe box (spheres and cubes) were scanned to test the basic functionality of the scanner. Angular increments appropriate for the servo motors and sensor capabilities were used to capture the spatial data.

Analysis

Upon analysis, discrepancies were noted between the scanned models and the actual dimensions of the objects, reflecting the limitations of ultrasonic sensors. The data resolution achieved provided a coarse representation of the objects, influenced by the sensor's limitations and the angular step sizes used. Limitations such as the ultrasonic sensor's wide cone angle and lower resolution impacted the precision of the measurements. Environmental factors, including ambient noise, also affected sensor readings, introducing additional challenges.

Conclusions

Despite the limitations, the project successfully demonstrated the principles of 3D scanning using basic components. It served as a valuable learning exercise, providing insights into the challenges of 3D scanning with crude sensors and highlighting the importance of precise components for accurate results. The creation of a functional 3D scanner that effectively integrated hardware and software components was a significant achievement. The Python GUI enhanced user interaction by providing control and visualization capabilities.

For future improvements, the use of more precise sensors like or LIDAR could be considered to enhance accuracy. Refinements in mechanical design and the implementation of advanced data processing techniques could further improve the system's performance.

10. Technical Summary of Project Outcomes and Achievements

The project resulted in the development of a functional prototype of a 3D scanner using basic sensors and components. Successful integration of mechanical, electrical, and software components was achieved, demonstrating the feasibility of creating a 3D scanner with readily available and inexpensive parts. Real-time visualization of the scanned data using Python-based tools aided in understanding the scanning process and improved user engagement.

Wireless communication between the microcontroller and the PC/Mac was established using BLE, eliminating the need for wired connections and increasing the system's flexibility. Challenges encountered included dealing with the inherent inaccuracy and wide cone angle of ultrasonic sensors. This was mitigated to some extent by implementing a tube attachment to focus the ultrasonic pulses. Mechanical inaccuracies were addressed by using ball bearings and gear coupling mechanisms to improve stability and reduce wear.

Data handling posed challenges due to noisy data and environmental interference. Basic filtering techniques were employed to manage these issues, although there is room for implementing more advanced data processing methods in the future.

The project provided valuable lessons on the feasibility of 3D scanning with basic components, the significant impact of sensor quality and mechanical precision on overall performance, and the complexities involved in software development for hardware integration. The experience gained sets a foundation for future projects and improvements in the field of low-cost 3D scanning solutions.

The project also involved researching the electronics required to handle the Pico without using a development board, leading to the design and construction of a custom circuit board. A MOSFET was also considered as a protective measure to prevent accidental damage to either the Pico or the PC/Mac during operation. (Notes provided below)

Example Command Sent from GU

```
# Sending start_scan command over BLE
import json
from bleak import BleakClient

command = {
    "command": "start_scan",
    "pan_range": [0, 180],
    "tilt_range": [0, 90],
    "sensor": "ultrasonic"
}

command_json = json.dumps(command)

async def send_command(address, command_json):
    async with BleakClient(address) as client:
        await client.write_gatt_char(CHARACTERISTIC_UUID, command_json.en
```

Pico BLE Initialization:

```
# MicroPython code for BLE setup on Pico
from bluetooth import BLE
import ujson

ble = BLE()
ble.active(True)

def on_write_callback(handle, data):
    command = ujson.loads(data.decode('utf-8'))
    # Process the command

ble_service_uuid = "12345678-1234-5678-1234-56789abcdef0"
ble_characteristic_uuid = "12345678-1234-5678-1234-56789abcdef1"

ble.gatts_register_services([
    (ble_service_uuid, [
        (ble_characteristic_uuid, BLE.WRITE | BLE.NOTIFY),
    ])
])

ble.gatts_set_callback(on_write_callback)
```

References

- MicroPython Documentation for Raspberry Pi Pico W
- Bleak Library Documentation for BLE Communication
- Matplotlib and Plotly Documentation for Data Visualization
- PyQt5 Documentation for GUI Development